Calculator with the Strategy Pattern

Assignment 5 | Software Design Patterns | Java 17
Student: **Kuatbay Merey** | Group: **SE-2537**
Course: ShP-2216 | Option B | Academic year: 2026-2027

1 Introduction

I chose a calculator with four interchangeable arithmetic algorithms: addition, subtraction, multiplication, and division. All accept two `double` operands and return a `double`. The Strategy pattern puts each algorithm behind a common interface, so the calculator can change its operation at runtime without a chain of operation-specific conditionals [1].

The project contains seven Java files and no external libraries. `Main` is the client, `Calculator` is the context, `OperationStrategy` is the strategy interface, and the four operation classes are concrete strategies. The client creates the strategies and selects one through a map. The context knows only the interface and delegates the calculation.

`Main` constructs one calculator before the menu loop. Selecting another operation calls `setStrategy` on that same object. Running `java -cp out Main --demo` demonstrates the change with the same operands:

```
10.0 + 5.0 = 15.0
10.0 - 5.0 = 5.0
10.0 * 5.0 = 50.0
10.0 / 5.0 = 2.0
```

The scope is a small console calculator. It uses binary floating-point arithmetic, so some decimal results are approximate. The console accepts finite operands, reports overflow, and handles invalid input. Division rejects both positive and negative zero. There is no expression parser, calculation history, or persistence.

2 UML class diagram

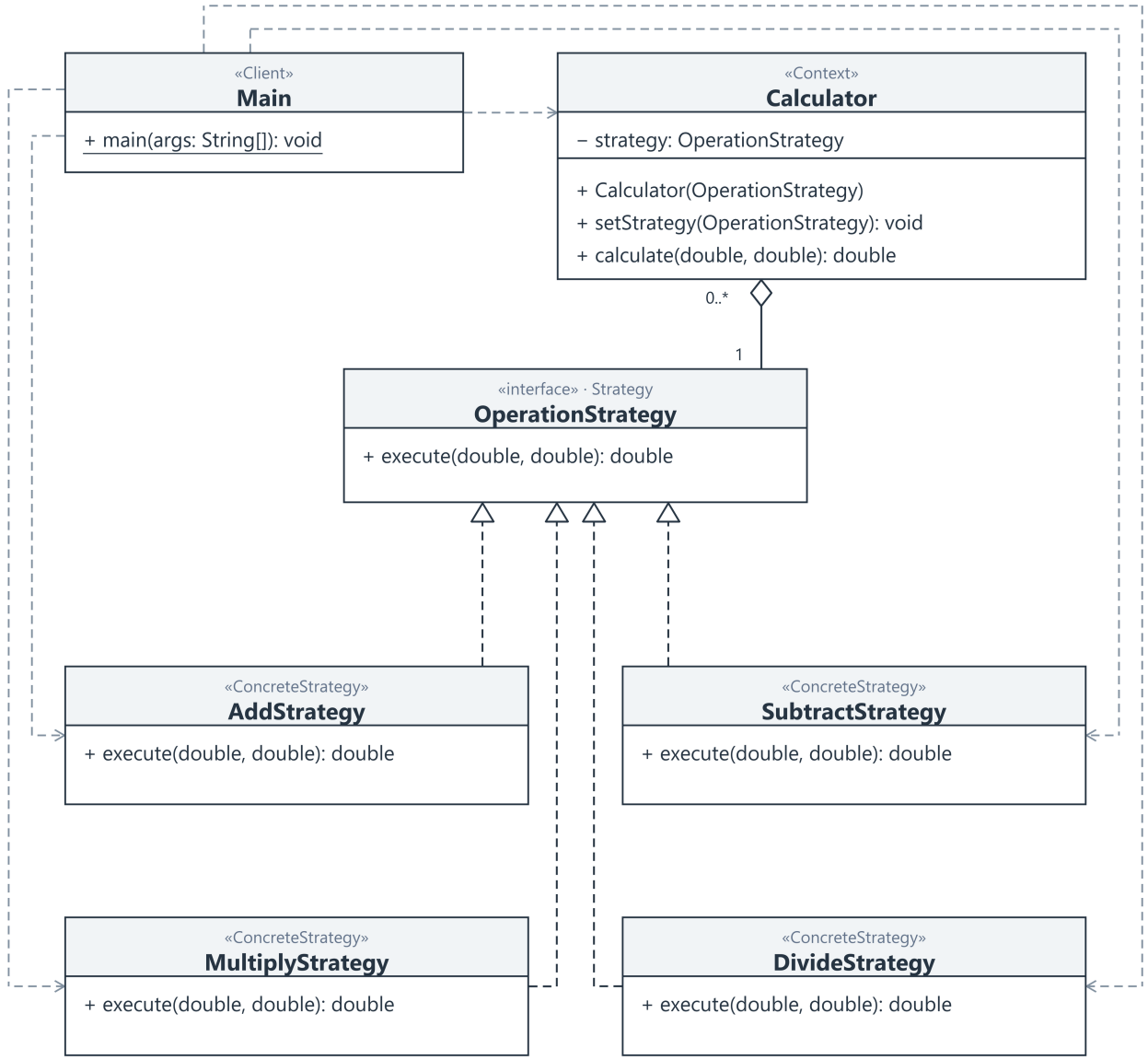


Figure 1. The implemented Strategy structure. Operation parameters are shown as types only. The diagram shows the public pattern operations and the context's private strategy field. The client's private strategy map and helpers, and Java library types, are omitted. Vector diagram and editable PlantUML source.

The **hollow diamond at** `Calculator` denotes shared aggregation: the context has exactly one active strategy, supplied from outside. Strategy objects exist independently, may be shared, and can be replaced. A filled composition diamond would imply stronger ownership and lifecycle responsibility that this design does not require [3].

The **dashed lines with hollow triangular heads at** `OperationStrategy` denote interface realization: every concrete operation implements `execute`. The **dashed dependency arrows from** `Main` show creation and use, rather than ownership. + means public and - means private. These relationships describe structure, not execution order.

3 Clean Code principles

Each numbered example includes an implementation excerpt and the reason for the design. The extension example in 3.5 is explicitly separate from the seven submitted source files.

3.1 Replace operation conditionals with polymorphism

```
// Calculator.java
public double calculate(double first, double second) {
    return strategy.execute(first, second);
}
```

Dynamic dispatch chooses the implementation. `Calculator` contains no `if`, `switch`, `instanceof`, or concrete strategy names. In the client, `OPERATIONS.get(choice)` finds the strategy and `calculator.setStrategy(strategy)` installs it. The remaining branches validate input, exit the menu, or handle errors; they do not select algorithms by type.

3.2 Use intention revealing names

```
// SubtractStrategy.java
public class SubtractStrategy implements OperationStrategy {
    @Override
    public double execute(double first, double second) {
        return first - second;
    }
}
```

The class name identifies the operation without a numeric mode or an explanatory comment. `first` and `second` make operand order explicit, which matters for subtraction and division. The same parameter names appear in every implementation.

3.3 Give each class one responsibility

```
// AddStrategy.java
public double execute(double first, double second) {
    return first + second;
}
```

The strategy performs one arithmetic algorithm. It does not read input, print a result, or choose the next strategy. `Main` owns the console interaction; `Calculator` owns delegation. Changing a prompt therefore does not change the arithmetic classes.

3.4 Depend on a small abstraction

```
// OperationStrategy.java
public interface OperationStrategy {
    double execute(double first, double second);
}
```

The interface exposes only the operation required by the context. `Calculator` stores an `OperationStrategy`, not an `AddStrategy` or a menu option. This keeps the context independent of concrete algorithm internals. Strategies also have no reference back to the context or to each other.

3.5 Extend behavior without changing the context

```
// Extension example, compiled separately during verification
class PowerStrategy implements OperationStrategy {
    @Override
    public double execute(double first, double second) {
        return Math.pow(first, second);
    }
}
```

A client can pass this class to the existing setter and evaluate `calculator.calculate(2.0, 3.0)` as `8.0`. A separate verification harness compiled and executed this extension against the submitted classes. `Calculator`, `OperationStrategy`, and the existing strategies required no changes. Adding a visible menu option still requires registering the class and updating the client menu; the Open/Closed benefit applies to the calculation code, not to every file in the application.

3.6 Encapsulate state and reject invalid configuration

```
// Calculator.java
private OperationStrategy strategy;

public void setStrategy(OperationStrategy strategy) {
    this.strategy = Objects.requireNonNull(strategy, "Strategy cannot be null.");
}
```

The strategy reference is private, and the setter preserves the invariant that one strategy is always available. The constructor performs the same null check. A failed setter call leaves the previous strategy intact. Separately, `DivideStrategy` checks `second == 0.0` before division and throws a clear `IllegalArgumentException`; the client displays the error and continues.

3.7 Verification

The complete project compiled with JDK 17.0.12 using `javac --release 17 -Xlint:all -Werror`, with no warnings. The automated demo matched the four results in Section 1 exactly.

A separate verification harness passed **22 API checks**, including arithmetic, switching on one context, both signed-zero divisors, null construction and replacement, strategy sharing, and the external power strategy. **22 console scenarios** passed, covering all operations in one session, invalid choices, malformed numbers, decimal and negative operands, non-finite inputs, overflow, recovery after division by zero, and EOF at each input stage. These checks ran during development; the delivered project keeps only the seven application source files.

Build and run instructions are in `README.md`. For a live demonstration, select Add with 10 and 5, then Subtract, Multiply, and Divide with the same operands. The context stays the same while the delegated algorithm changes.

4 Strategy vs State

Strategy fits this problem because the client deliberately chooses an arithmetic algorithm for a calculation. Addition does not transition itself to subtraction, and no operation represents a stage in the calculator's lifecycle. State instead models behavior that depends on an object's internal state and its transitions, such as a document moving from draft to published [2]. Both patterns can delegate through an interface, but this design changes an independently selected algorithm, rather than following a state transition model.

5 Conclusion

The calculator can switch operations at runtime and accept a new algorithm without changing its context or existing algorithms. The cost is several small classes and client configuration for each exposed operation. For a four-operation calculator, direct arithmetic would be shorter, but separating the algorithms makes the Strategy roles visible and keeps each class easy to explain. The project therefore keeps the required pattern while avoiding frameworks and extra architectural layers.

6 GitHub repository

https://github.com/bossmerej-ship-it/SDP-5

The repository contains the Java sources, a short README, this Markdown report, a formatted PDF copy, and the UML assets. The commit history separates the arithmetic context and strategies, the console client and demo, and the report and diagrams.

References and assistance

[1] Refactoring.Guru. Strategy. Used for the pattern roles, interchangeable algorithms, and runtime selection.

[2] Refactoring.Guru. State. Used for the comparison with state-dependent behavior and transitions.

[3] Object Management Group. Unified Modeling Language 2.5.1. Used for the distinction between aggregation, composition, realization, and dependency.

Requirements: *Assignment 5 Strategy Pattern 2026-2027*, supplied assignment brief. The previous assignment repository was used as a reference for project size, structure, and report style.

Development assistance: OpenAI Codex assisted with implementation, report drafting, diagram preparation, and verification.